

Developer_Manual.md

Backend Database Relational Schemas and Scraping Pipelines

This architectural manual serves as the engineering blueprint for the KNPC Market Intelligence System. It details the relational data store designs, data collection ingestion pipelines, error isolation frameworks, and downstream analytical handling modules.

1. DATABASE SCHEMA SPECIFICATIONS (MARKET_DATA.DB)

The relational storage structure runs on an embedded SQLite engine. Tables enforce uniqueness constraints to guarantee data purity over repetitive pipeline execution sweeps:

```
CREATE TABLE IF NOT EXISTS crude_prices (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    price_date TEXT,  
    benchmark TEXT,      -- e.g., 'Brent', 'WTI', 'Oman', 'KEC'  
    price REAL,  
    previous_price REAL,  
    daily_change REAL,  
    daily_change_pct REAL,  
    unit TEXT,  
    source TEXT,  
    collected_at TEXT,  
    UNIQUE(price_date, benchmark, source)  
);
```

```
CREATE TABLE IF NOT EXISTS product_prices (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    price_date TEXT,  
    market TEXT,  
    product TEXT,        -- e.g., 'Naphtha', 'Gasoil 10ppm'  
    price REAL,  
    previous_price REAL,  
    daily_change REAL,  
    daily_change_pct REAL,  
    status TEXT,         -- 'collected' or 'manual_required'  
    notes TEXT,  
    proxy_type TEXT,  
    benchmark_basis TEXT,  
    collected_at TEXT,  
    UNIQUE(price_date, product)  
);
```

```
CREATE TABLE IF NOT EXISTS product_collection_attempts (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    attempt_date TEXT,  
    market TEXT,  
    product TEXT,  
    source TEXT,  
    status TEXT,         -- 'success', 'warning', 'error'  
    message TEXT,
```

```
        collected_at TEXT
    );
```

2. DATA COLLECTION INGESTION ENGINE (COLLECTORS.PY)

The ingestion pipeline is designed with defensive fault-isolation boundaries to minimize web scraping vulnerability. It uses a browser-spoofing header dictionary to prevent anti-bot response drops, and incorporates multi-ticker fallback arrays for volatile regional instruments.

For instance, when pulling pricing data for localized metrics like Oman crude, the collector loop iterates through an explicit list of alternative symbols (e.g., Yahoo financial tickers) until a valid payload returns, guaranteeing data continuity.

3. INGESTION ERROR ISOLATION & COMPLIANCE BADGING

If an external target web portal shifts its CSS DOM structure, times out over the network, or fails to publish on a regional holiday, the collectors.py engine intercepts the resulting exception to protect the application runtime. Instead of halting execution, it records a deterministic fallback record:

```
* price Column Value      --> Writes explicit SQL NULL
* source Reporting Column --> Writes 'Source system didn't publish'
* status Processing Flag  --> Labels record as 'manual_required'
```

Downstream, the analytics.py aggregation engine filters out these null rows from chronological line charts, preventing skewed metrics. Concurrently, the app.py rendering grid reads the status code and wraps the data cell in a prominent alert badge, instructing operators to supply a verified override price via the UI ledger panel.

4. EXCEL PROCESSING MATRIX ENGINE (EXPORTER.PY)

The file exporter handles reporting compliance by dynamically building a multi-sheet spreadsheet workbook utilizing OpenPyXL. The execution sequence runs as follows:

1. Queries chronological tracking history for crude records and refined proxy rows.
2. Generates two high-level analytical calculation summary frames: Benchmark_Summary and Product_Summary.
3. Compiles the master flat-file database tables into All_Benchmarks and All_Products tabs.
4. Iterates through every unique asset identifier and provisions a dedicated worksheet named after the specific asset, allowing users to drill down into historical records.